ESGI
école supérieure de génie informatique

Séance 1 - Concevoir une API HTTP

Au programme

- Positionnement (30 min, en tout début de séance)
- Le problème : une API n’est pas une liste de fonctions exposées en HTTP
- L’échelle de maturité de Richardson : les niveaux, et pourquoi viser le 2
- Ressources et collections : le modèle
- Concevoir les URIs : ce qui se fait, ce qui ne se fait pas
- Les verbes et leur sémantique : sûr, idempotent
- Les familles de codes de statut, et les pièges (201, 400 vs 422, 404 vs 403)
- Rappel HTTP fin : négociation de contenu, en-têtes conditionnels
- Le fil rouge : 3 sujets, une API par étudiant

© 2026 Nicolas FAESSEL

Le problème : exposer des fonctions vs exposer des ressources

Le réflexe : prendre les fonctions du code et leur coller une URL.

```
POST /createReservation
POST /getReservationsByUser
POST /cancelReservationById
POST /updateSeatCount
```

> Chaque équipe invente ses noms, ses conventions, ses codes de retour. Rien n'est prévisible : le client doit lire la doc pour **chaque** appel.

L'approche ressource : un petit nombre de **noms** (les ressources), un petit nombre de **verbes** standard (ceux d'HTTP), une **sémantique connue d'avance**.

© 2026 Nicolas FAESSEL

L’échelle de maturité de Richardson

Une grille pour évaluer **à quel point une API s’appuie sur HTTP**.

Niveau	Ce qu’on ajoute	Exemple (billetterie)
0	rien : HTTP sert de tunnel, tout en `POST`	`POST /api` + `{ "action": "reserver" }` ou `POST /createReservation`
1	des **ressources**, chacune son URI	`POST /reservations/42` pour lire, annuler, modifier
2	les **verbes** et **codes** HTTP	`GET` / `DELETE /reservations/42`, `201`, `404`, `409`
3	l’**hypermédia** : liens vers la suite	`"links": { "next": "/seances?page=2" }`

Leonard Richardson (2008), popularisé par Martin Fowler, « Richardson Maturity Model » (2010).

© 2026 Nicolas FAESSEL

Richardson : pourquoi monter les niveaux

Plus on monte, plus le client (et le cache, le proxy, la bibliothèque HTTP) comprend l'API **sans lire sa doc**.

Niveau	Ce que le client y gagne
1	une adresse stable par chose : on la désigne, on la relie, on la retrouve dans les logs
2	une sémantique connue : `GET` se cache et se rejoue, `DELETE` rejoué ne change rien, `4xx` ≠ `5xx`
3	les actions possibles sont dans la réponse : le client suit les liens au lieu de fabriquer les URIs

> **Dans ce module : le niveau 2 partout**, plus une touche de niveau 3 : les liens de pagination (`links.next` , séance 2). En TP, l'échelle sert de grille pour relire votre `api-design.md` .

© 2026 Nicolas FAESSEL

Ressources et collections

Une **ressource** est une chose que l'API expose et qu'on peut nommer : une réservation, un adhérent, une séance de cinéma.

Une **collection** est une ressource qui contient d'autres ressources : l'ensemble des réservations, l'ensemble des adhérents.

```
/reservations            la collection
/reservations/42         un élément de la collection
/seances/7/reservations  les réservations d'une séance (sous-collection)
```

> On modélise **les choses**, pas les actions. « Annuler une réservation » n'est pas une ressource : c'est un changement d'état de la ressource `reservations/42` .

© 2026 Nicolas FAESSEL

Concevoir les URIs

- **Noms au pluriel** pour les collections : `/ouvrages`, `/adherents`, `/cours`
- **Pas de verbe dans l'URI** : `/reservations/42`, pas `/getReservation?id=42`
- **Identifiant stable** dans le chemin : `/reservations/42`
- **Relations** en sous-ressource quand la vie de l'une dépend de l'autre : `/ouvrages/128/exemplaires`
- **Filtres, tri, pagination** en paramètres de requête, pas dans le chemin : `/ouvrages?auteur=herbert&tri=titre&page=2`

> Le chemin identifie **une** ressource ou **une** collection. Tout ce qui restreint, ordonne ou pagine une collection passe par la *query string*.

© 2026 Nicolas FAESSEL

Les verbes et leur sémantique

Verbe	Sûr	Idempotent	Usage
`GET`	oui	oui	lire une ressource ou une collection
`POST`	non	non	créer un élément dans une collection, déclencher un traitement
`PUT`	non	oui	remplacer une ressource entière à un emplacement connu
`PATCH`	non	non	modifier partiellement une ressource (idempotent seulement si conçu pour)
`DELETE`	non	oui	supprimer une ressource

Sûr = ne modifie rien. **Idempotent** = rejouer la requête à l'identique laisse le serveur dans le même état qu'un seul appel. On s'en ressert en séance 4 pour la concurrence.

© 2026 Nicolas FAESSEL

Les codes de statut : les familles

Famille	Sens	Exemples
`2xx`	succès	200 OK, 201 Created, 204 No Content
`3xx`	redirection / cache	304 Not Modified
`4xx`	le client s'est trompé	400, 401, 403, 404, 409, 422
`5xx`	le serveur a échoué	500, 502, 503

> La famille se lit d'un coup d'oeil. Un client bien écrit branche sa logique de reprise sur le **premier chiffre** avant de regarder le détail.

© 2026 Nicolas FAESSEL

Les codes de statut : les choix qui reviennent

- `201 Created` après un `POST` réussi + en-tête `Location` vers la ressource créée
- `204 No Content` quand il n'y a rien à renvoyer (souvent après `DELETE`)
- `400 Bad Request` : la requête est mal formée (JSON invalide, champ absent)
- `422 Unprocessable Content` : la requête est bien formée mais **sémantiquement** invalide (date de fin avant la date de début)
- `404 Not Found` vs `403 Forbidden` : « ça n'existe pas » vs « ça existe mais pas pour toi » - parfois on renvoie 404 exprès pour ne pas révéler l'existence

© 2026 Nicolas FAESSEL

Un échange complet

```
POST /seances/7/reservations HTTP/1.1
Content-Type: application/json

{ "sieges": ["A12", "A13"], "client": "c-901" }
```

```
HTTP/1.1 201 Created
Location: /reservations/42
Content-Type: application/json

{ "id": 42, "seance": 7, "sieges": ["A12", "A13"], "statut": "confirmee" }
```

Le client crée dans une **collection** (`POST` sur `/seances/7/reservations`), le serveur répond **où** la ressource vit maintenant (`Location`). Aucune doc nécessaire pour comprendre l'échange.

© 2026 Nicolas FAESSEL

HTTP : la négociation de contenu

- `Content-Type` décrit le corps **qu'on envoie** : `Content-Type: application/json`
- `Accept` décrit ce que le client **sait recevoir** : `Accept: application/json`
- Le serveur choisit une représentation et l'annonce dans le `Content-Type` de la réponse

> Erreur classique : lire `Content-Type` sur une requête `GET` sans corps. C'est `Accept` qui porte la demande du client.

Les **en-têtes conditionnels** (`ETag` , `If-Match` , `If-None-Match`) servent au cache et à la concurrence : on les ouvre en séance 4, mais on les croise dès les premières réponses.

© 2026 Nicolas FAESSEL

Le fil rouge : une API par étudiant

À partir d'aujourd'hui, chacun construit **sa** propre API HTTP, sur un sujet choisi en fin de séance.

- Chaque séance applique la notion du jour à **votre** API
- Le livrable de chaque TP est **produit en séance** et versionné dans votre dépôt
- **Projet individuel** : un sujet différent par étudiant
- Le module tient en 15h : **aucun développement n'est attendu hors séance**
- Le langage est **libre** : vous codez dans celui que vous maîtrisez déjà

© 2026 Nicolas FAESSEL

Les 3 sujets

#	Sujet	Ce qui le caractérise
1	**Bibliothèque municipale**	catalogue, exemplaires, emprunts ; recherche et pagination d'un gros catalogue
2	**Billetterie de cinéma**	séances, plan de salle, réservation de places ; concurrence sur les sièges
3	**Salle de sport**	cours, plannings récurrents, réservation de créneaux ; quotas et liste d'attente

> Domaines volontairement distincts : chacun rencontrera un **piège** différent (séance 4). Détail des attendus : `evals/projet-sujets.md`.

© 2026 Nicolas FAESSEL

Choisir son sujet

- **Un étudiant par sujet** : trois sujets, trois étudiants
- Choix **immédiat** ; en cas de double préférence, on tranche en séance
- Le sujet choisi est acté dans le `README` de votre dépôt, **avant la fin de la séance**

> Vous garderez ce sujet jusqu'à la soutenance (séance 10). Choisissez un domaine dont les règles métier vous parlent : vous allez les modéliser pendant 10 séances.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Modèle de ressources + initialisation du projet

Objectif : initialiser votre dépôt, choisir votre sujet, et concevoir le modèle de ressources et le jeu d'URIs de votre API.

- Handout : `supports/tp/session-01.md`
- **Livrable attendu** : dépôt initialisé (`README` , arborescence, lancement documenté) + `docs/api-design.md` contenant
  - la liste des ressources et collections de votre sujet (au moins 4 ressources liées)
  - le tableau URI x verbe x code de statut attendu pour les opérations principales
  - 2 à 3 choix de conception justifiés en une phrase
- Sujet fil rouge **acté et commité**

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 2 - Collections

Au programme

- Conception du schéma, puis kit de données de votre sujet
- CRUD : les cinq opérations d'une collection
- `PUT` vs `PATCH` : remplacer ou modifier
- Le problème d'une collection qui grossit
- Pagination offset et enveloppe de réponse
- Filtrage, tri stable, recherche
- Aparté : le problème N+1 côté REST

© 2026 Nicolas FAESSEL

CRUD : les opérations d'une collection

Opération	Requête	Réponse en cas de succès
Créer	`POST /ouvrages`	`201 Created` + `Location`
Lister	`GET /ouvrages`	`200 OK` + collection
Lire un élément	`GET /ouvrages/128`	`200 OK` + ressource
Remplacer	`PUT /ouvrages/128`	`200 OK` ou `204 No Content`
Modifier	`PATCH /ouvrages/128`	`200 OK`
Supprimer	`DELETE /ouvrages/128`	`204 No Content` ; `404` si absent

> Cinq verbes, une collection : un client qui connaît ce tableau sait déjà utiliser l'API sans lire de doc.

© 2026 Nicolas FAESSEL

`PUT` vs `PATCH`

`PUT` **remplace** la ressource entière. Les champs absents du corps sont considérés comme effacés.

```
PUT /adherents/42
Content-Type: application/json

{ "nom": "Bob", "email": "bob@ex.fr", "actif": true }
```

`PATCH` **modifie** les champs fournis, laisse les autres tels quels.

```
PATCH /adherents/42
Content-Type: application/json

{ "email": "nouveau@ex.fr" }
```

> Envoyer un corps partiel en `PUT` est un bug fréquent : les champs manquants sont perdus.

© 2026 Nicolas FAESSEL

Une collection qui grossit

`GET /ouvrages` sur un catalogue de 40 000 titres :

- réponse énorme, lente à produire, lente à transférer, lente à parser
- le client n’a besoin que des 20 premiers
- la base souffre à chaque appel

> Une collection ne se renvoie **jamais** en entier. On la découpe en pages, et on donne au client de quoi demander la suivante.

© 2026 Nicolas FAESSEL

Pagination offset

Le client demande une tranche par sa position :

```
GET /ouvrages?page=3&limit=20
```

ou, équivalent, `?offset=40&limit=20`.

- simple à implémenter (`LIMIT ... OFFSET ...` en SQL)
- permet de sauter directement à la page N
- **dérive** si des éléments sont insérés ou supprimés pendant la navigation : on peut sauter ou revoir un élément d'une page à l'autre

La pagination par **curseur**, qui corrige cette dérive, est vue en séance 4.

© 2026 Nicolas FAESSEL

L’enveloppe de réponse

Une collection paginée renvoie les éléments **et** de quoi se repérer :

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "data": [ { "id": 128, "titre": "Dune" }, "..." ],
  "pagination": { "page": 3, "limit": 20, "total": 402 },
  "links": { "next": "/ouvrages?page=4&limit=20", "prev": "/ouvrages?page=2&limit=20" }
}
```

> Choisissez une forme d’enveloppe et tenez-la sur **toutes** vos collections. Le client écrit son code de pagination une seule fois.

© 2026 Nicolas FAESSEL

Filtrage

Les critères de restriction passent en paramètres de requête, combinables :

```
GET /ouvrages?auteur=herbert&disponible=true&annee_min=1960
```

- chaque paramètre est optionnel
- absent = pas de filtre sur ce critère
- un paramètre inconnu : l'ignorer, ou renvoyer `400` - à décider et à documenter

© 2026 Nicolas FAESSEL

Tri, et le piège du tri instable

```
GET /ouvrages?tri=titre&ordre=asc
```

Si plusieurs ouvrages ont le même titre, leur ordre relatif n'est **pas garanti** d'un appel à l'autre. En paginant, un élément peut apparaître page 2 puis page 3.

> **Tri stable** : ajouter toujours une clé de départage unique, par exemple `tri=titre,id`. L'ordre devient déterministe, la pagination cohérente.

© 2026 Nicolas FAESSEL

Recherche simple

Un paramètre `q` pour une recherche plein texte basique :

```
GET /ouvrages?q=épice&limit=10
```

- porte sur un ou plusieurs champs (titre, auteur, résumé) - à documenter
- se combine avec le filtrage et la pagination
- la recherche avancée (pertinence, facettes) est hors périmètre de ce module

© 2026 Nicolas FAESSEL

Le problème N+1

Lister 20 ouvrages, puis pour chacun charger ses exemplaires : 21 allers-retours pour une page.

```
SELECT * FROM ouvrages ORDER BY titre, id LIMIT 20       -- 1 fois
SELECT * FROM exemplaires WHERE ouvrage_id = ?           -- 20 fois
```

Remède : 2 requêtes, quelle que soit la taille de la page.

```
SELECT * FROM exemplaires WHERE ouvrage_id IN (128, 131, ...)   -- 1 fois
```

- pas de jointure naïve : `LIMIT 20` compterait 20 lignes jointes, pas 20 ouvrages
- côté contrat : lien vers la relation, ou relation embarquée à la demande (`?include=exemplaires`)

> On le retrouve en séance 7 : en GraphQL, chaque resolver ne voit qu'un parent, le N+1 y apparaît naturellement.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - CRUD et collection navigable

Objectif : implémenter le modèle de ressources de votre sujet et rendre une collection paginable, filtrable et triable.

- Handout : `supports/tp/session-02.md`
- Kit de données, remis après 10 min de conception : `supports/kits/<sujet>/`
- **Livrable attendu** :
  - esquisse du modèle de données dans `docs/api-design.md`
  - API qui démarre et répond ; CRUD complet sur au moins 2 ressources
  - une collection paginée (offset) + filtrable + triable, avec enveloppe de réponse
  - début de la collection de requêtes versionnée (`requests/`)
  - stratégie de pagination notée en une phrase dans le `README`

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 3 - Échouer proprement

Un format d’erreur unique, une validation systématique, des tests qui le prouvent

Au programme

- Le problème : chaque erreur a sa forme
- `application/problem+json` (RFC 9457)
- Un handler d’erreur central
- Valider l’entrée sur toutes les écritures
- `400` vs `422`, le détail par champ
- Tests d’intégration HTTP : ce qu’on teste

© 2026 Nicolas FAESSEL

Le problème : des erreurs qui ne se ressemblent pas

Dans une même API, au fil du code :

```
HTTP/1.1 404 Not Found
"Ouvrage introuvable"
```

```
HTTP/1.1 400 Bad Request
{ "error": "invalid", "champ": "email" }
```

```
HTTP/1.1 500 Internal Server Error
{ "message": "Error: ECONNREFUSED", "stack": "..." }
```

> Le client ne peut pas traiter les erreurs de façon générique : il doit coder un cas par endpoint. Et la dernière fuit des détails d'implémentation.

© 2026 Nicolas FAESSEL

`application/problem+json` (RFC 9457)

Un format standard pour toutes les erreurs d'une API HTTP :

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

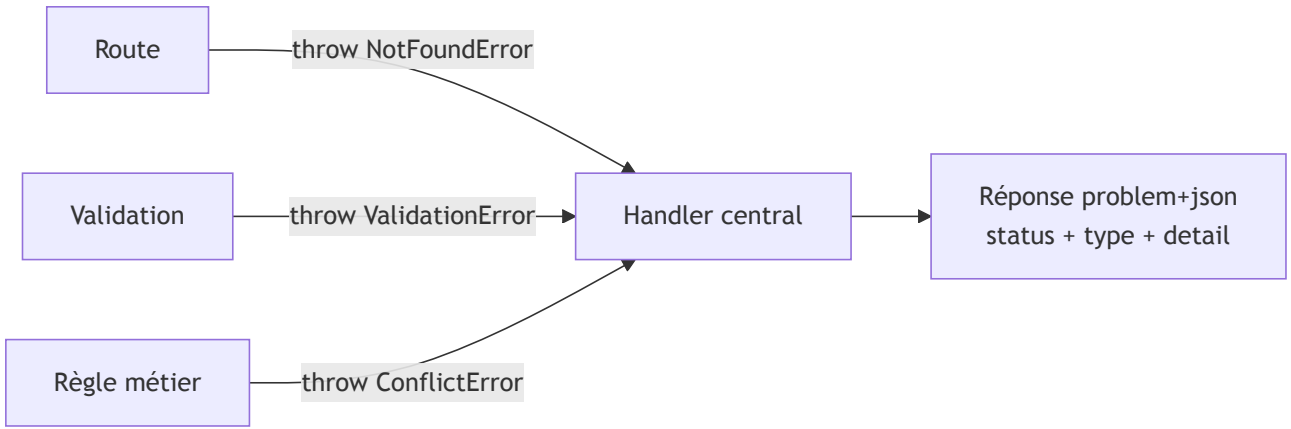
{
  "type": "https://api.exemple.fr/errors/siege-indisponible",
  "title": "Le siège demandé n'est plus disponible",
  "status": 409,
  "detail": "Le siège A12 de la séance 7 a été réservé entre-temps.",
  "instance": "/seances/7/reservations"
}
```

> `type` identifie la **catégorie** d'erreur (une URL stable), `title` la résume, `detail` décrit **ce cas précis**. On peut ajouter des champs d'extension (`seat` , `errors` …).

© 2026 Nicolas FAESSEL

Un handler d’erreur central

Toutes les erreurs de l’API passent par **une seule** fonction qui les met au format `problem+json`.



> Les routes lèvent des erreurs typées, jamais la réponse d’erreur : un seul endroit à faire évoluer.

© 2026 Nicolas FAESSEL

Valider l’entrée sur toutes les écritures

Toute requête `POST`, `PUT`, `PATCH` est validée **avant** d’atteindre la logique métier :

- présence des champs requis
- types attendus (une date est une date, un entier un entier)
- valeurs de bord (quantité > 0, email bien formé, énumération respectée)
- champs inconnus : rejetés ou ignorés, au choix documenté

> Un schéma déclaratif (JSON Schema ou l’équivalent de votre framework) décrit ces règles une fois, à côté de la route. Pas de `if` de validation éparpillés.

© 2026 Nicolas FAESSEL

`400` vs `422`

- `400 Bad Request` : la requête est **mal formée**. Le serveur ne peut même pas la lire. JSON invalide, en-tête manquant.
- `422 Unprocessable Content` : la requête est **bien formée** mais **sémantiquement** invalide. La date de fin est avant la date de début ; l'adhérent référencé n'existe pas.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{ "type": ".../validation", "title": "Requête invalide", "status": 422,
  "errors": [ { "field": "date_fin", "message": "doit être après date_debut" } ] }
```


© 2026 Nicolas FAESSEL

Le détail par champ

Quand la validation échoue sur plusieurs champs, renvoyez **tous** les problèmes d'un coup :

```
{
  "type": "https://api.exemple.fr/errors/validation",
  "title": "Requête invalide",
  "status": 422,
  "errors": [
    { "field": "email", "message": "format invalide" },
    { "field": "quantite", "message": "doit être supérieur à 0" }
  ]
}
```

> Un client (ou un formulaire) peut afficher chaque erreur à sa place. Renvoyer les champs un par un oblige l'utilisateur à corriger, renvoyer, recommencer.

© 2026 Nicolas FAESSEL

Tester une API : intégration vs unitaire

Test unitaire	Test d'intégration HTTP
une fonction isolée	l'API démarrée, une vraie requête HTTP
dépendances simulées	la chaîne réelle : routing, validation, handler d'erreur
« la fonction calcule bien »	« `POST /x` avec ce corps renvoie `422` et ce `problem+json` »

> Pour une API, le test d'intégration est le plus rentable : il vérifie le **contrat** que voit le client, pas l'implémentation.

© 2026 Nicolas FAESSEL

Ce qu’on teste

Pour chaque route, au moins :

- **le cas nominal** : requête valide → statut et corps attendus
- **les cas d’erreur** : corps invalide → `422` ; ressource absente → `404` ; conflit → `409`

```
# scénario : créer une réservation sur une séance qui n'existe pas
POST /seances/999/reservations   →   404 Not Found (problem+json)
```

> Ces scénarios vivent dans votre collection de requêtes **et** dans la suite de tests : les mêmes appels, joués à la main ou automatiquement.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Erreurs, validation, tests

Objectif : un format d'erreur unique sur toute l'API, une validation sur toutes les écritures, une suite de tests d'intégration.

- Handout : `supports/tp/session-03.md`
- **Livrable attendu** :
  - format `problem+json` unique, produit par un handler central
  - validation d'entrée sur toutes les routes d'écriture, avec détail par champ (une erreur capturée)
  - suite de tests d'intégration qui tourne : cas nominaux CRUD / pagination + 2 à 3 cas d'erreur
  - collection de requêtes enrichie des cas d'erreur

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 4 - Deux requêtes au même moment

Idempotence, `409`, requêtes conditionnelles, et le piège de votre sujet

Au programme

- Rappel : verbes sûrs et idempotents
- Le `POST` rejoué, et la clé d'idempotence
- `409 Conflict` : refuser une opération incohérente
- Requêtes conditionnelles : `ETag`, `If-None-Match`, `If-Match`
- Le piège de votre sujet, décliné
- Le reproduire : deux requêtes concurrentes

© 2026 Nicolas FAESSEL

Rappel : sûr et idempotent

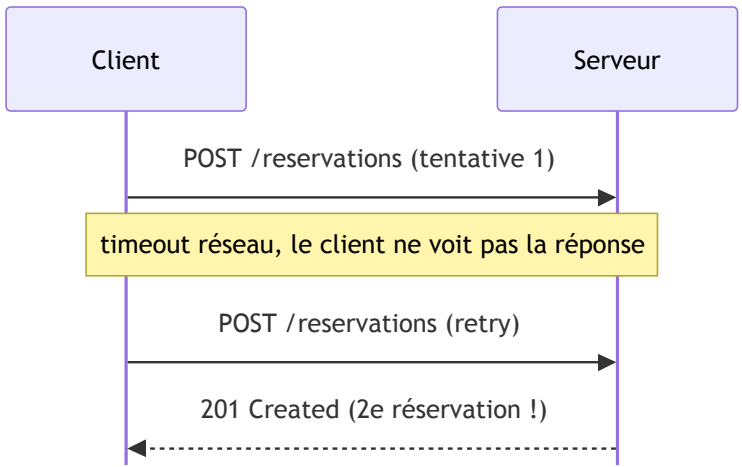
- **Sûr** : ne modifie rien (`GET`)
- **Idempotent** : rejouer à l'identique laisse le serveur dans le même état (`GET` , `PUT` , `DELETE`)
- **Ni l'un ni l'autre** : `POST`

> `PUT /adherents/42` rejoué trois fois : l'adhérent 42 a la même valeur qu'après un seul appel. `POST /adherents` rejoué trois fois : **trois** adhérents créés.

© 2026 Nicolas FAESSEL

Le `POST` rejoué

Un double-clic, un retry réseau, un client impatient : la même création part deux fois.



> Le client voulait **une** réservation. Il en a deux, et n'en connaît qu'une.

© 2026 Nicolas FAESSEL

La clé d’idempotence

Le client génère une clé unique par **intention** et la répète sur les retries :

```
POST /reservations
Idempotency-Key: 7b2c1a90-4e...
Content-Type: application/json

{ "seance": 7, "sieges": ["A12"] }
```

- première fois : le serveur traite, stocke le résultat sous cette clé
- rejeu avec la même clé : le serveur renvoie **le résultat stocké**, sans re-créer

> C’est le client qui fournit la clé : lui seul sait que deux requêtes sont la **même** intention.

© 2026 Nicolas FAESSEL

`409 Conflict`

L'opération est bien formée, autorisée, mais **incohérente avec l'état courant** :

```
POST /seances/7/reservations
{ "sieges": ["A12"] }
```

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{ "type": ".../siege-indisponible", "title": "Siège déjà réservé",
  "status": 409, "detail": "Le siège A12 a été pris entre-temps." }
```

> `409` n'est pas `422`. Avec `422`, la requête est mauvaise dans l'absolu ; avec `409`, elle serait valide à un autre moment, mais l'état a changé.

© 2026 Nicolas FAESSEL

Requêtes conditionnelles : `ETag`

Le serveur étiquette chaque version d'une ressource :

```
GET /ouvrages/128
→ 200 OK
  ETag: "v7"
```

Au prochain `GET`, le client propose son étiquette :

```
GET /ouvrages/128
If-None-Match: "v7"
→ 304 Not Modified   (corps vide, rien n'a changé)
```

> Économie de bande passante : le serveur ne renvoie le corps que si la version a changé.

© 2026 Nicolas FAESSEL

`If-Match` : concurrence optimiste

Sur une **écriture**, le client dit « je modifie la version que j'ai lue » :

```
PUT /ouvrages/128
If-Match: "v7"
```

- la ressource est toujours en `"v7"` → la modification passe, l'`ETag` devient `"v8"`
- quelqu'un l'a modifiée depuis (`"v8"`) → `412 Precondition Failed`, la modification est refusée

> Pas de verrou posé à l'avance. On tente, et on échoue proprement si la ressource a bougé. Le client recharge et réessaie.

© 2026 Nicolas FAESSEL

Le piège de votre sujet

Sujet	Piège caractéristique
Bibliothèque	pagination **par curseur** + tri stable sur un gros catalogue mouvant ; double emprunt d'un exemplaire (`409`)
Cinéma	réservation **concurrente de sièges** : hold temporaire, confirmation idempotente, `409` propre
Salle de sport	plannings **récurrents** + fenêtre de réservation + **quota** par adhérent ; bascule en liste d'attente

> C'est le coeur de la note de robustesse à la soutenance. Détail par sujet : `evals/projet-sujets.md` .

© 2026 Nicolas FAESSEL

Pagination par curseur (tous les sujets, piège du sujet 1)

L'offset dérive quand la collection bouge. Le **curseur** encode « où on s'est arrêté » :

```
GET /ouvrages?tri=titre,id&limit=20
→ { "data": [...], "links": { "next": "/ouvrages?tri=titre,id&limit=20&after=RHVuZXwxMjg" } }
```

- `after` = position du dernier élément renvoyé (encodée)
- insertions et suppressions ailleurs dans la liste : sans effet sur la suite de la navigation
- prix à payer : on ne peut plus sauter à « la page 500 »

© 2026 Nicolas FAESSEL

Reproduire le piège

Un test qui lance **deux requêtes concurrentes** sur la ressource sensible :

```
# en parallèle, deux clients sur le dernier siège
POST /seances/7/reservations  { "sieges": ["A12"] }   → 201 Created
POST /seances/7/reservations  { "sieges": ["A12"] }   → 409 Conflict
```

> Le livrable de la séance, c'est **ce scénario reproductible** et la preuve que votre API réagit correctement : un gagnant, un `409` propre, aucun état incohérent.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Le piège de concurrence de votre sujet

Objectif : traiter et démontrer le piège caractéristique de votre sujet ; ajouter le cache conditionnel.

- Handout : `supports/tp/session-04.md`
- **Livrable attendu** :
  - le piège du sujet traité, démontré par un scénario reproductible (2 requêtes concurrentes), comportement attendu capturé
  - une collection paginée par curseur, cohérente malgré une insertion pendant le parcours
  - `ETag` + requête conditionnelle (`If-None-Match` en lecture, `If-Match` en écriture) sur au moins une ressource
  - section « **Stratégie face au piège** » dans `docs/api-design.md` : stratégie retenue + « Pourquoi REST ici »

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 5 - La spec comme source de vérité

Décrire l'API dans un contrat lisible par les humains et les machines, et vérifier que le code le respecte

Au programme

- Code d’abord vs contract-first
- Anatomie d’un document OpenAPI 3.1
- `components/schemas` et `$ref`
- Décrire les erreurs `problem+json`
- Valider la spec, générer la doc
- Contract testing : « valide » n’est pas « fidèle »

© 2026 Nicolas FAESSEL

Deux façons de faire

Code d'abord : on code l'API, puis un outil génère une spec à partir des annotations. La spec suit le code, avec du retard, et ment souvent.

Contract-first : on écrit (ou on met à jour) la spec, elle sert de référence, le code s'y conforme et des tests le vérifient.

> Dans ce module, votre API existe déjà (séances 1 à 4). En séance 5, vous écrivez la spec qui la décrit **fidèlement**, puis vous maintenez les deux en phase.

© 2026 Nicolas FAESSEL

Anatomie d’un document OpenAPI 3.1

```yaml
openapi: 3.1.0
info:
  title: API Bibliothèque
  version: "1.0.0"
paths:
  /ouvrages/{id}:
    get:
      summary: Lire un ouvrage
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        "200":
          description: L'ouvrage demandé
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Ouvrage" }
        "404":
          $ref: "#/components/responses/ProblemNotFound"
```


© 2026 Nicolas FAESSEL

`components/schemas` et `$ref`

Une ressource est décrite **une fois**, puis référencée partout :

```yaml
components:
  schemas:
    Ouvrage:
      type: object
      required: [id, titre]
      properties:
        id:     { type: integer }
        titre:  { type: string }
        auteurs: { type: array, items: { type: string } }
```

> `$ref: "#/components/schemas/Ouvrage"` dans la réponse du `GET`, dans le corps du `POST`, dans la liste paginée. Une seule définition à faire évoluer.

© 2026 Nicolas FAESSEL

Décrire les erreurs `problem+json`

Le format d'erreur de la séance 3 se déclare comme n'importe quel schéma :

```yaml
components:
  schemas:
    Problem:
      type: object
      properties:
        type:   { type: string }
        title:  { type: string }
        status: { type: integer }
        detail: { type: string }
  responses:
    ProblemNotFound:
      description: Ressource introuvable
      content:
        application/problem+json:
          schema: { $ref: "#/components/schemas/Problem" }
```


© 2026 Nicolas FAESSEL

Valider la spec, générer la doc

Valider : un linter vérifie que le document est un OpenAPI 3.1 correct - `$ref` qui pointent dans le vide, types invalides, réponses sans description.

Générer la doc : Redoc, Swagger UI ou Scalar transforment la spec en page navigable, servie par l'API (`/docs`) ou publiée en statique.

> La doc n'est plus un document Word à part qui se périme. Elle **est** la spec, rendue lisible.

© 2026 Nicolas FAESSEL

Contract testing

Un test qui compare une **vraie réponse** de l'API au **schéma** de la spec :

```
GET /ouvrages/128   →   200 OK
{ "id": 128, "titre": "Dune", "note": 4.5 }
```

```
✗ Le champ "note" n'est pas déclaré dans le schéma Ouvrage
```

> Soit le code a un champ en trop, soit la spec en a un en moins. Le test force à trancher, à chaque build.

© 2026 Nicolas FAESSEL

« Valide » n’est pas « fidèle »

- une spec **valide** : le document est un OpenAPI 3.1 syntaxiquement correct
- une spec **fidèle** : elle décrit ce que l’API fait **réellement**

> Une spec valide qui décrit une API imaginaire est pire que pas de spec : les clients codent contre un contrat que personne ne respecte. C’est le contract testing qui garantit la fidélité.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Spécifier l'API en OpenAPI 3.1

Objectif : écrire la spec qui décrit votre API réelle, publier la doc, ajouter un test de contrat.

- Handout : `supports/tp/session-05.md`
- **Livrable attendu** :
  - `openapi.yaml` valide décrivant l'API réelle : routes CRUD + pagination + réponses d'erreur + `ETag` / `Idempotency-Key` là où présents
  - doc générée publiée (URL ou fichier)
  - au moins un test qui vérifie une réponse contre le schéma ; écart spec / code identifié et corrigé
  - stratégie face au piège relue et complétée (checkpoint projet en ouverture de séance)

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 6 - Sécuriser l'API

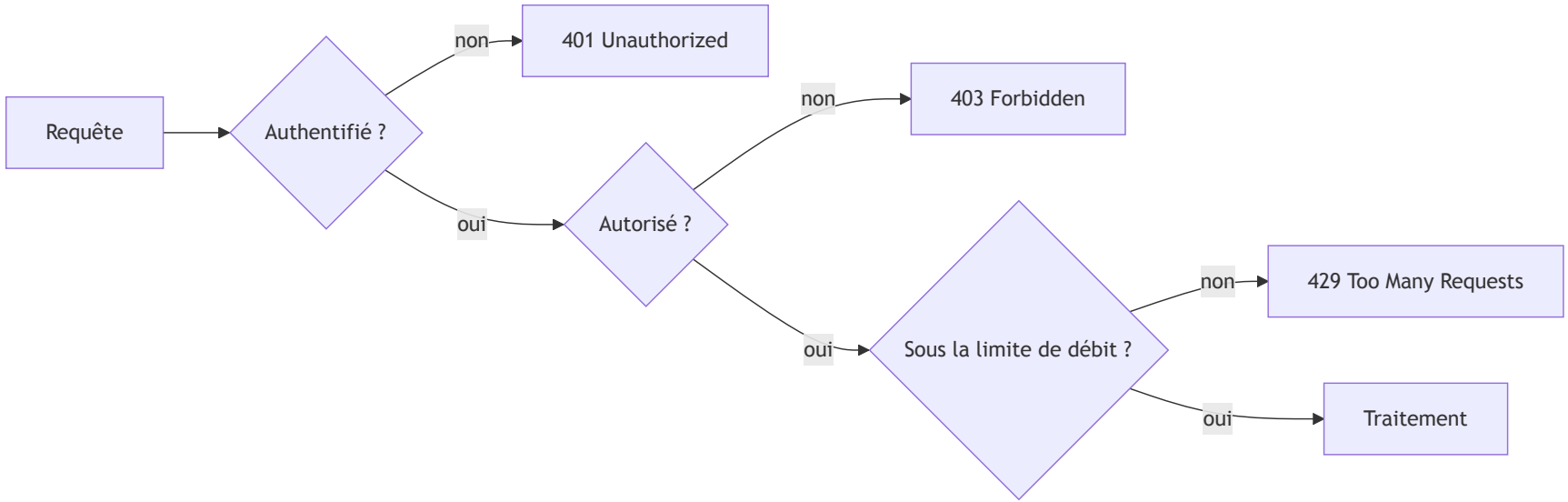
Qui es-tu, as-tu le droit, à quel rythme : fermer l'API sans la rendre inutilisable

Au programme

- Authentification par token : `Authorization: Bearer`
- Anatomie d’un JWT
- `401` vs `403`
- Autorisation par rôle ou scope
- Rate-limiting : `429` et `Retry-After`
- CORS et gestion des secrets

© 2026 Nicolas FAESSEL

Trois questions distinctes



> Trois portes successives : identité, droits, débit. On les traite séparément.

© 2026 Nicolas FAESSEL

Authentification par token

Le client obtient un token à un endpoint dédié, puis le présente à chaque requête :

```
POST /auth/token
{ "client_id": "...", "client_secret": "..." }
→ 200 OK
  { "access_token": "eyJhbGci...", "token_type": "Bearer", "expires_in": 3600 }
```

```
GET /reservations
Authorization: Bearer eyJhbGci...
```

> Le token remplace l'envoi des identifiants à chaque appel. Il a une **durée de vie** : passé `expires_in`, il faut en redemander un.

© 2026 Nicolas FAESSEL

Anatomie d’un JWT

Trois parties séparées par des points, encodées en base64url :

```
eyJhbGciOiJIUzI1NiJ9  .  eyJzdWIiOiJjLTkwMSIsInJvbGUiOiJhZGhlcmVudCIsImV4cCI6MTcuLi59  .  3vT8s...
      header                          payload (claims)                                      signature
```

```
{ "sub": "c-901", "role": "adherent", "exp": 1717000000 }
```

> Le payload est **lisible** par tous (pas chiffré). C’est la **signature** qui garantit qu’il n’a pas été modifié : le serveur la vérifie avec sa clé. `exp` = date d’expiration, vérifiée à chaque requête.

© 2026 Nicolas FAESSEL

`401` vs `403`

- `401 Unauthorized` : « je ne sais pas qui tu es ». Token absent, invalide, ou expiré. Le client doit s'authentifier (ou rafraîchir son token).
- `403 Forbidden` : « je sais qui tu es, mais tu n'as pas le droit ». Se ré-authentifier n'y changera rien.

> Confondre les deux envoie le client dans une boucle : il re-tente l'authentification alors que le problème est un manque de droits.

© 2026 Nicolas FAESSEL

Autorisation par rôle ou scope

Après avoir identifié le client, on vérifie ce qu'il a le droit de faire :

- **par rôle** : `adherent`, `coach`, `admin` - des ensembles de permissions
- **par scope** : `reservations:read`, `reservations:write` - des permissions fines portées par le token

```
PATCH /reservations/42
token.sub = "c-901",  reservation 42 appartient à "c-555"
→ 403 Forbidden  (un adhérent ne modifie que ses propres réservations)
```

> Au moins **une** règle métier de ce type dans votre projet, avec un test qui la vérifie.

© 2026 Nicolas FAESSEL

Rate-limiting

Limiter le nombre de requêtes par client et par fenêtre de temps :

```
GET /ouvrages
→ 429 Too Many Requests
  Retry-After: 30
  Content-Type: application/problem+json

  { "type": ".../rate-limit", "title": "Trop de requêtes", "status": 429 }
```

> `Retry-After` indique au client combien de temps attendre. Un client correct le respecte au lieu de marteler l'API.

© 2026 Nicolas FAESSEL

CORS, en une slide

CORS contrôle quels **sites web** (origines) peuvent appeler votre API **depuis un navigateur** :

- requête « simple » : le navigateur envoie, puis vérifie l'en-tête `Access-Control-Allow-Origin` de la réponse
- requête « preflight » : pour `PUT`, `DELETE`, en-têtes custom → le navigateur envoie d'abord un `OPTIONS` pour demander la permission

> CORS protège l'utilisateur d'un **autre site**, pas votre API. `curl`, Postman, un serveur : aucun n'est concerné par CORS. Ce n'est **pas** un mécanisme d'autorisation.

© 2026 Nicolas FAESSEL

Les secrets ne sont pas du code

- clé de signature JWT, mot de passe de base, clé d'API tierce : **jamais** commités
- lus depuis des **variables d'environnement**
- un fichier `.env` **ignoré par git**, accompagné d'un `.env.example` sans valeurs

```
.gitignore       →  .env
.env.example     →  JWT_SECRET=
                    DB_URL=
```

> Un secret commité une fois reste dans l'historique git même après suppression. Il faut le considérer comme compromis et le changer.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Fermer l'API

Objectif : protéger les endpoints, poser une règle d'autorisation testée, configurer CORS et un rate-limit, sortir les secrets du dépôt.

- Handout : `supports/tp/session-06.md`
- **Livrable attendu** :
  - endpoints protégés (sans token → `401` capturé)
  - au moins une règle d'autorisation testée (mauvais rôle → `403` capturé, test dédié)
  - CORS configuré (preflight qui répond) ; rate-limit sur au moins une route (`429` + `Retry-After` capturé)
  - aucun secret dans le dépôt (`.env` ignoré, `.env.example` fourni) ; spec OpenAPI mise à jour avec le *security scheme*

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 7 - Faire évoluer, comparer

Changer un contrat sans casser les clients ; confronter REST à GraphQL sur la même ressource

Au programme

- Le contrat a des clients
- Changement rétro-compatible vs cassant
- Stratégies de versioning
- Déprécier proprement : `Deprecation`, `Sunset`
- GraphQL : un endpoint, le client décrit ce qu'il veut
- Le problème N+1 en GraphQL ; REST vs GraphQL

© 2026 Nicolas FAESSEL

Le contrat a des clients

Une fois l'API publiée, du code ailleurs en dépend. Vous ne le contrôlez pas.

> Renommer un champ, supprimer une route, rendre un paramètre obligatoire : ces changements **cassent** les clients existants du jour au lendemain.

La question n'est pas « comment je change mon API » mais « comment je la change **sans casser ce qui tourne** ».

© 2026 Nicolas FAESSEL

Rétro-compatible vs cassant

Changement	Rétro-compatible ?
Ajouter un champ **optionnel** à une réponse	oui
Ajouter une nouvelle route	oui
Ajouter une valeur à une énumération de sortie	oui (prudence)
Rendre un paramètre optionnel obligatoire	**non**
Renommer ou supprimer un champ	**non**
Changer le type d’un champ	**non**
Resserrer une règle de validation	**non**

© 2026 Nicolas FAESSEL

Stratégies de versioning

Stratégie	Exemple	Compromis
Dans l'URI	`/v1/ouvrages`	visible, simple à router, « pollue » les URLs
En-tête custom	`API-Version: 2`	URLs stables, moins découvrable
Média-type	`Accept: application/vnd.api.v2+json`	rigoureux, verbeux, peu utilisé

> Le versioning dans l'URI est le plus répandu. L'essentiel est d'en avoir **un**, explicite, dès la v1 - pas de l'ajouter en catastrophe à la v2.

© 2026 Nicolas FAESSEL

Déprécier proprement

Avant de retirer quelque chose, on prévient, longtemps à l'avance :

```
GET /v1/ouvrages
→ 200 OK
  Deprecation: @1796083200
  Sunset: Mon, 01 Mar 2027 00:00:00 GMT
  Link: <https://api.exemple.fr/docs/migration-v2>; rel="deprecation"
```

`Deprecation` : « déprécié depuis cette date » (RFC 9745, `@` + horodatage Unix). `Sunset` : « ça s'arrête à cette date ». Plus la doc de migration. Les anciens tests continuent de passer jusqu'à la date annoncée.

© 2026 Nicolas FAESSEL

GraphQL : un endpoint, une requête décrite

Le client envoie ce qu'il veut, exactement :

```
query {
  ouvrage(id: 128) {
    titre
    exemplaires { code, disponible }
  }
}
```

```
{ "data": { "ouvrage": { "titre": "Dune",
  "exemplaires": [ { "code": "EX-1", "disponible": true } ] } } }
```

Un seul `POST /graphql`. Pas de sur-récupération (on demande 2 champs, on reçoit 2 champs), pas de sous-récupération (ouvrage **et** exemplaires en un aller-retour).

© 2026 Nicolas FAESSEL

Le schéma GraphQL

Typé, déclaré côté serveur ; chaque champ est résolu par une fonction (*resolver*) :

```
type Ouvrage {
  id: ID!
  titre: String!
  exemplaires: [Exemplaire!]!
}
type Query {
  ouvrage(id: ID!): Ouvrage
}
```

> Le resolver de `exemplaires` est appelé pour **chaque** ouvrage du résultat. C'est là que revient le problème N+1.

© 2026 Nicolas FAESSEL

Le problème N+1, encore

```
query { ouvrages(limit: 20) { titre, exemplaires { code } } }
```

- 1 requête pour la liste des 20 ouvrages
- puis 20 requêtes, une par ouvrage, pour charger ses exemplaires

> Même problème qu'en REST (séance 2). Solution : un **dataloader** regroupe les 20 demandes d'exemplaires en **une** requête `WHERE ouvrage_id IN (...)`, exécutée une fois par tick.

© 2026 Nicolas FAESSEL

REST vs GraphQL

Critère	REST	GraphQL
Forme de la réponse	fixée par le serveur	choisie par le client
Nombre d’aller-retours	souvent plusieurs	un
Cache HTTP (`ETag` , CDN)	natif par URL	difficile (un seul endpoint, `POST`)
Couplage client / serveur	faible	le client connaît le schéma
Outillage réseau (`curl` , proxy)	direct	passe par une couche

Pas de gagnant absolu. GraphQL brille pour des clients aux besoins variés (mobile, web, partenaires) ; REST reste plus simple à cacher et à observer.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Versioning et prototype GraphQL

Objectif : mettre en place le versioning et un changement rétro-compatible ; exposer une ressource en GraphQL et démontrer le N+1.

- Handout : `supports/tp/session-07.md`
- **Livrable attendu** :
  - versioning explicite + un changement rétro-compatible documenté (avant / après + test de non-régression) + un en-tête `Deprecation`
  - mini-prototype GraphQL : une ressource exposée, une query qui traverse une relation, N+1 démontré
  - section « **REST vs GraphQL** » amorcée dans `docs/api-design.md`

© 2026 Nicolas FAESSEL

ESGI
école supérieure de génie informatique

Séance 9 - gRPC et Protocol Buffers

Un troisième style, orienté service à service : contrats binaires, RPC, streaming

> Séance enseignée **après le CC**. Hors périmètre de l'évaluation écrite ; utile pour le comparatif de `docs/api-design.md` et la soutenance.

Au programme

- Appeler des méthodes distantes avec un contrat fort
- Le fichier `.proto` : `message`, `service`, `rpc`
- Les champs numérotés et l'évolution d'un schéma
- RPC unaire vs streaming
- Service à service vs API publique
- REST vs GraphQL vs gRPC : le tableau de décision

© 2026 Nicolas FAESSEL

Un autre modèle

REST expose des **ressources** manipulées par des verbes HTTP. gRPC expose des **méthodes** qu'on appelle presque comme des fonctions locales :

```
reponse = client.RechercherOuvrages(requete)
```

- le contrat est un fichier `.proto`, partagé entre client et serveur
- les deux côtés génèrent leur code à partir de ce fichier
- les messages circulent en **binaire**, sur HTTP/2

© 2026 Nicolas FAESSEL

Le fichier `.proto`

```proto
syntax = "proto3";

message RechercheRequest {
  string q = 1;
  int32 limit = 2;
}

message Ouvrage {
  int32 id = 1;
  string titre = 2;
  repeated string auteurs = 3;
}

message RechercheResponse {
  repeated Ouvrage resultats = 1;
}

service Catalogue {
  rpc RechercherOuvrages(RechercheRequest) returns (RechercheResponse);
}
```


© 2026 Nicolas FAESSEL

Les champs numérotés

Chaque champ a un **numéro** (`= 1` , `= 2`), pas seulement un nom. C'est le numéro qui est encodé dans le binaire, pas le nom.

> Conséquence pour l'évolution : on peut **renommer** un champ librement (le numéro ne change pas), **ajouter** un champ avec un nouveau numéro (les vieux clients l'ignorent). On ne **réutilise jamais** un numéro abandonné.

C'est une compatibilité ascendante et descendante intégrée au format, là où REST demande de la discipline (séance 7).

© 2026 Nicolas FAESSEL

RPC unaire vs streaming

Type	Forme	Exemple
Unaire	1 requête → 1 réponse	`RechercherOuvrages`
Server streaming	1 requête → flux de réponses	suivre les places libres d'une séance
Client streaming	flux de requêtes → 1 réponse	envoyer un lot d'événements
Bidirectionnel	deux flux simultanés	chat, synchronisation

> Le streaming est natif, sur une seule connexion HTTP/2. En REST il faut SSE ou WebSocket (hors module).

© 2026 Nicolas FAESSEL

Binaire : rapide, mais opaque

```
POST /catalogue.Catalogue/RechercherOuvrages HTTP/2
content-type: application/grpc+proto

<octets binaires>
```

- compact et rapide à sérialiser : bon pour du trafic interne à fort volume
- **illisible** au `curl`, non inspectable dans l'onglet Réseau d'un navigateur
- pas accessible directement depuis un navigateur (nécessite un proxy `grpc-web`)

> Le gain de performance se paie en observabilité et en outillage. À assumer.

© 2026 Nicolas FAESSEL

Service à service vs API publique

Bon terrain pour gRPC : deux services que **vous** contrôlez, qui s'appellent souvent, où le couplage fort au schéma est acceptable et la performance compte.

Mauvais terrain : une API publique, consommée par des tiers, depuis des navigateurs, qu'on veut pouvoir explorer au `curl` et cacher via un CDN.

> Beaucoup de systèmes réels font les deux : REST au bord (public), gRPC à l'intérieur (entre services).

© 2026 Nicolas FAESSEL

REST vs GraphQL vs gRPC

Critère	REST	GraphQL	gRPC
Format	texte (JSON)	texte (JSON)	binaire (proto)
Transport	HTTP/1.1+	HTTP, un endpoint	HTTP/2
Navigateur	direct	direct	via proxy
Cache HTTP	natif	difficile	non
Streaming	non (SSE/WS)	limité	natif
Couplage	faible	moyen	fort (schéma partagé)
Terrain	API publique	clients variés	service à service

© 2026 Nicolas FAESSEL

Votre comparatif

Complétez le comparatif de `docs/api-design.md` : pour **votre** sujet, comparez les trois styles.

- pourquoi REST est le bon choix par défaut pour votre API publique
- où GraphQL apporterait quelque chose (quels clients, quels besoins)
- où gRPC aurait du sens (quel appel interne, entre quels composants)

> Comparatif complet attendu en fin de séance. C’est une pièce de la soutenance.

© 2026 Nicolas FAESSEL

TP

© 2026 Nicolas FAESSEL

TP - Un service gRPC minimal

Objectif : écrire un `.proto`, implémenter un RPC unaire qui réexpose une opération de votre sujet, répondre depuis un client de test.

- Handout : `supports/tp/session-09.md`
- **Livrable attendu** :
  - un service gRPC minimal (1 `.proto` + 1 RPC unaire, éventuellement 1 stream) qui répond à un client de test
  - **comparatif REST / GraphQL / gRPC complété** dans `docs/api-design.md`

© 2026 Nicolas FAESSEL